Privacy-Preserving Recommendation Systems

A Survey of Cryptographic Approaches

Mainak Sarkar Shrasti Dwivedi Aditya Anand Shravan Agrawal

Literature Survey

CS670: Cryptographic Techniques for Privacy Preservation
Indian Institute of Technology Kanpur

Outline

Part I

The problem, the pipeline, and a taxonomy

Why this is a problem

Recommender systems are built from the most revealing data users produce: what they watched, read, bought and clicked.

The data is not just sensitive — it is *identifying*

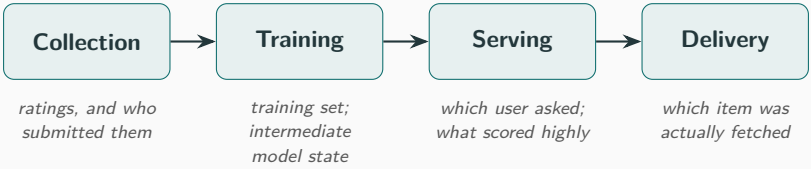
Narayanan & Shmatikov (2008) de-anonymised the Netflix Prize dataset using IMDb as background knowledge, recovering records of known users and their "apparent political preferences and other potentially sensitive information".

A large body of work asks: can a recommender be built so that its operator never sees the underlying data?

Answers come from several communities — **secure MPC, PIR, oblivious memory, federated learning, differential privacy** — with different vocabularies, threat models, and *parts of the system* they protect.

The recommendation pipeline

The survey's organising device: a deployed recommender splits into four stages.



Different lines of work protect different subsets of these stages.

Consequence

A pipeline is only as private as its *weakest stage*. Training under secure computation achieves little if the item is then fetched in the clear — the fetch reveals the recommendation.

Utility constrains the cryptography

Recommendation quality is measured by ranking metrics over a held-out set — nDCG@k and recall@k.

- A private recommender that recommends *badly* has solved nothing.
- The constraint propagates into protocol design: **fixed-point arithmetic**, approximation of non-linear functions, and reduced precision must not perturb the ranking.

The core tension

Cryptographic primitives compute over rings and finite fields; recommendation computes over the *reals*. Bridging that gap is where much of the practical cost is incurred.

Reporting quality *alongside* cost is therefore the norm in this literature.

A taxonomy along four axes

Systems are routinely compared as if alternatives — but they often protect *different stages* under *different assumptions*. Four axes make comparison honest:

1. Trust model

Who must be trusted? Single server / non-colluding servers / federated / trusted hardware.

2. Privacy notion

What does "private" mean? Cryptographic (simulation) / differential privacy / anonymity.

3. Pipeline stage protected

Collection / Training / Serving / Delivery — the axis the literature is least explicit about.

4. Adversary

Semi-honest vs. malicious; how many parties may be corrupted, and whether fixed in advance.

Key idea: these axes are **not orderable by strength** — they are different assumptions, not better or worse ones.

Placing the systems (condensed)

Group / example	Trust model	Notion	Stage	Adversary
Recommendation systems				
Pirsona	s+1 pairwise n.c.	crypto	C,T,D	semi-honest
Nudge	3, ≤ 1 corrupt	crypto	C,T,S	semi-honest
FedMF	federated + HE	crypto	C,T	semi-honest
McSherry–Mironov	single server	DP	T	—
Private retrieval / nearest-neighbour				
Tiptoe	single server	crypto	S	semi-honest
Pacmann	single server	crypto	S	semi-honest
Compass	single server	crypto	S	compromised svr.
Wally	server-side	DP	S	semi-honest
Private information retrieval				
SimplePIR/Spiral	single server	crypto	D	semi-honest
Chor et al. (multi-server)	multi-server n.c.	crypto	D	semi-honest
Oblivious memory (primitives)				
Floram/Duoram/Prac	2–3 party	crypto	—	semi-honest

Stages: **C**ollection, **T**raining, **S**erving, **D**elivery. Full 32-system table appears as Table 1 in the report.

Three observations from the table

1. The stage column is uneven.
   Training and serving are well populated; *delivery* is addressed almost entirely by the PIR line in isolation. Only PIRSONA spans collection, training and delivery together.

2. Cost is bought with assumptions, consistently.
   The cheapest protocols in each group assume non-collusion. Removing the assumption (single-server) normally costs something.

3. Semi-honest is the norm at scale.
   Every recommendation system in the table is semi-honest; malicious-security exceptions are narrow and not full pipelines.

Part II

Cryptographic primitives, and private training

Cryptographic building blocks

Systems are assembled from a small set of primitives. What matters is each one's cost profile — communication, computation, or rounds.

Secret sharing

Split a value across parties. Additive (2-party) or replicated 2-of-3. Addition is free; multiplication needs correlated randomness / rounds.

Garbled circuits

Two parties evaluate a Boolean circuit, constant rounds. Cost is bandwidth $\propto$ circuit size $\Rightarrow$ binds on data-oblivious algorithms.

Homomorphic encryption

Compute on ciphertexts. Enables *single-server* constructions, removing non-collusion — at a computational cost.

FSS / distributed point functions

Share a *function*. Keys are logarithmic in domain size (*compression*) — makes private read/write layers practical.

Private information retrieval (PIR)

Fetch record j without the server learning j. Multi-server (non-colluding) or single-server (computational).

Oblivious RAM / DORAM

Hide *access patterns*. Distributed ORAM for the two-party / MPC setting (Floram, Duoram).

Fixed-point arithmetic dominates the cost

Primitives compute over rings; recommendation computes over the reals. A real v is stored as $\lfloor v \cdot 2^t \rfloor$.

- Addition is free; a product is $2t$-scaled and must be truncated back.
- Repeated multiplication needs normalisation to stop values growing.
- Both are **non-linear** and expensive under secret sharing.

Recurring theme

Linear algebra is cheap; the non-linear steps are not. SecureML contributes "MPC-friendly alternatives to non-linear functions such as sigmoid and softmax"; frameworks (ABY, MP-SPDZ) exist precisely because different operations are cheapest under different sharings.

Private training (1): the garbled-circuit era

Every system here computes the same object: a low-rank factorisation $\mathbf{U} \approx \mathbf{A} \cdot \mathbf{B}$ of a rating matrix nobody may see.

Nikolaenko et al. (2013)

First practical system — gradient descent *inside a garbled circuit* across two non-colluding servers.

Cost profile: the computation must be data-oblivious, so the circuit is sized by the *whole rating matrix* — NUDGE reports this family needing over four orders of magnitude more communication than secret-sharing approaches.

Lesson: a circuit-based approach pays for every operation in circuit size. Secret sharing inverts that relationship.

Private training (2): HE and the secret-sharing line

Homomorphic encryption — applied *surgically*

Removes non-collusion by keeping data encrypted at one server, but FHE of a training loop is expensive. FedMF uses HE only on the *one channel that leaks* (gradient uploads) rather than the whole computation.

Secret-sharing MPC — where the field advanced furthest

- SecureML (2 servers): established the pattern; the hard part was non-linearities, not linear algebra.
- SecureNN (3 parties): protocols for ReLU, maxpool, etc.; 6×–113× faster inference than prior work.

Private training (3): Nudge — the current reference point

NUDGE's central move is to change the algorithm, not just optimise it.

- Under 2-of-3 replicated sharing, a matrix–vector product is very cheap.
- So it replaces gradient descent with power iteration — mostly matrix–vector products, few non-linear steps.
- User embeddings **A** stay secret-shared; item embeddings **B** are revealed in the clear (a design choice with consequences).

Headline result — privacy need not cost accuracy

On Netflix (½M users, 10K items), 3×192-core servers: trains in 50 min, scoring **nDCG@20 = 0.29** — on par with non-private matrix factorisation (0.31 for neural). Privacy holds "against an adversary that compromises the entire secret state of one server".

Where each training family stops

Family	Stops at...
Garbled circuits	**scale** — circuit size grows with the data.
Homomorphic enc.	**cost** — buys single-server, applied to specific channels.
Secret-sharing MPC	**the trust assumption** — fast, but needs 2–3 non-colluding semi-honest parties.
Federated learning	**the guarantee** — gradient uploads leak without an added mechanism.
Differential privacy	**utility & scope** — protects the output, not the computation.

A limit none of them addresses: every system ends with a *model or scores*. None delivers the recommended *item* — that is Part III.

Part III

Delivering the item, and composing the pipeline

The obstruction: indices are data-dependent

Two problems remain after training: compute a ranking without revealing the user's profile, and fetch the item without revealing which it was. Both reduce to **retrieval** (studied as *private nearest-neighbour search*).

Why retrieval is hard to hide

Retrieval in the clear is fast *because of indices* — a k-d tree, an inverted file, a navigable graph. An index is useful precisely because the path through it *depends on the query* — which is exactly what obliviousness forbids.

The organising trade-off: scan the whole corpus every query (cost linear in corpus size), *or* keep the index and hide the traversal (oblivious memory + overhead).

Retrieval approaches (1): linear scan

Sanns — the origin point

Secure k-NN as "backbone of recommender systems"; optimised linear scan + a sublinear clustering variant, from HE + DORAM + garbled circuits. Top-k selection itself is a protocol-design problem.

Tiptoe — largest-scale linear approach

Reduces private full-text search to private nearest-neighbour search with semantic embeddings; linearly-homomorphic encryption. Strongest trust model here: "based on cryptography alone; no hardware enclaves or non-colluding servers".

45-server cluster, 360M web pages: 145 core-sec compute, 56.9 MiB comms, 2.7 s end-to-end. Ranks best result at position 7.7 (vs. 2.3 neural, 6.7 tf-idf).

Retrieval approaches (2): index-based and relaxed

Sublinear / index-based

PACMANN: moves work *to the client* — private graph-ANN using the preprocessing PIR paradigm. 90% quality of non-private ANN; up to 62% less compute on 100M vectors.

COMPASS: keeps the index server-side, hides traversal with oblivious memory (white-box ORAM co-design). "Latencies within a second"; privacy of "data, queries, and results, even if the server is compromised".

Relaxing the guarantee

PANTHER: single-server; co-designs PIR + SS + GC + HE. ANNS query on 10M points in 18 s, 284 MB — 7.8× faster, 20× more compact than prior.

WALLY: relaxes the *privacy notion* to **differential privacy** to break the linear-scan barrier. Batches queries from many clients, each adding "fake queries". Trust model conventional; notion is DP, not crypto.

ASHAROV et al.: similar-patient genomic search — shows the answer itself is sensitive; protecting the computation does not protect the answer.

Systems spanning multiple stages

Pirsona — collection, training *and* delivery

Treats PIR and collaborative filtering — "seemingly antithetical primitives" — together. Delivery uses multi-server PIR; collection is folded into delivery: consumption histories are extracted from the query traffic itself (the pipeline becomes a *cycle*). Price: strong $s+1$-server pairwise non-collusion.

Nudge — collection, training and serving

Users secret-share ratings; servers train and compute scores in shared form. Explicit *non-goal*: relies on "other means (Apple's private relay, Tor, or PIR)" for the user to fetch items. Protected up to the point the user *acts*.

The composition problem: trust models must agree, party roles need not match, and the interface between stages carries information.

What the field has and has not built

Reading Table 1 *by column*:

- The **training** stage is well served (Part II).
- The **serving** stage is well served.
- Delivery is addressed almost entirely by the PIR literature — *in isolation from any recommender*.

The gap in one sentence

Private-retrieval systems take a corpus and embeddings *as given*; private-training systems produce models and stop. PIRSONA spans both but *predates* that line *almost* in its entirety: PIRSONA 2021, against TIPTOE 2023 and PACMANN/COMPASS/PANTHER/WALLY 2024–2025 — though SANNS (2020) is the one exception.

We do not claim that composing these lines is straightforward, nor that no unpublished work has done so — only that among the systems surveyed here, the two halves are solved *separately*.

Part IV

How the field measures itself, and what remains open

How this literature evaluates itself

Datasets

MovieLens and Netflix Prize for recommendation; MS MARCO / large vector corpora for retrieval. No shared benchmark spans both halves of the pipeline.

Quality metrics

nDCG@k, recall@k. Best practice: report quality *against a non-private baseline* (Nudge: 0.29 vs. 0.31; Tiptoe: rank 7.7 vs. 2.3 neural).

Cost metrics & network model

Three quantities trade off: **local computation, bandwidth, round complexity**. Which dominates depends entirely on the deployment — a figure without its network model, hardware and dataset is not a result.

Why cross-paper comparison is unsound

Systems differ *simultaneously* in dataset, scale, hardware, network model, adversary, non-collusion count and pipeline stage. The survey reports each system in **its authors' own terms** and declines to build a league table.

Open problems (from the surveyed authors themselves)

1. Malicious security at scale. Nearly every system at realistic scale is semi-honest; the exceptions are narrower than a full pipeline.
2. Reducing the non-collusion assumption. The fastest protocols assume non-collusion (about organisations, not mathematics). Single-server at competitive cost is open.
3. The delivery stage. Addressed by PIR in isolation; composing it with a privately trained recommender is unanswered.

Open problems — the deepest one

Leakage through the recommendation itself

Cryptographic privacy guarantees nothing leaks *beyond the agreed output*. But it says nothing about *what the output itself reveals*. A recommendation is a statement about the user, delivered to a system that observes whether they act on it.

- NUDGE reveals item embeddings and scores *by design* — not a flaw, but information no in-protocol cryptography would remove.
- Differential privacy addresses exactly this class of leak, at a utility cost.
- This is not a protocol failure — no improvement in protocol design addresses it.

The direction we intend to pursue

Our course project

Engages with the composition of a privately trained recommender with a private delivery mechanism — the gap most visible in Table 1.

Concretely, the open questions we take from the survey:

- What does it cost to compose a modern private-retrieval backend with a privately trained recommender, given the composition obstacles?
- Does PIRSONA's collection-from-delivery mechanism transfer to a *modern* training substrate?

Conclusion

Privacy-preserving recommendation is not one problem but four, one per pipeline stage.

- **Training** and **serving** are in good shape — crypto privacy costs compute and communication, *not* recommendation quality (Nudge: nDCG on par with MF).
- **Delivery** is served by a mature PIR literature that is, with one exception, *not connected to any recommender*.
- The exception — PIRSONA — shows collection, training and delivery *can* be addressed together, but its training core has since been superseded.
- The leak carried by the **recommendation itself** survives any protocol — the most fundamental open problem.

Thank you